

Is Machine Learning Necessary for Cloud Resource Usage Forecasting?

Georgia
Christofidi

IMDEA Software Institute
Madrid, Spain
georgia.christofidi@imdea.org

Konstantinos
Papaioannou

IMDEA Software Institute
Madrid, Spain
konstantinos.papaioannou@imdea.org

Thaleia Dimitra
Doudali

IMDEA Software Institute
Madrid, Spain
thaleia.doudali@imdea.org

ABSTRACT

Robust forecasts of future resource usage in cloud computing environments enable high efficiency in resource management solutions, such as autoscaling and overcommitment policies. Production-level systems use lightweight combinations of historical information to enable practical deployments. Recently, Machine Learning (ML) models, in particular Long Short Term Memory (LSTM) neural networks, have been proposed by various works, for their improved predictive capabilities. Following this trend, we train LSTM models and observe high levels of prediction accuracy, even on unseen data. Upon meticulous visual inspection of the results, we notice that although the predicted values seem highly accurate, they are nothing but versions of the original data shifted by one time step into the future. Yet, this clear shift seems to be enough to produce a robust forecast, because the values are highly correlated across time. We investigate time series data of various resource usage metrics (CPU, memory, network, disk I/O) across different cloud providers and levels, such as at the physical or virtual machine-level and at the application job-level. We observe that resource utilization displays very small variations in consecutive time steps. This insight can enable very simple solutions, such as data shifts, to be used for cloud resource forecasting and deliver highly accurate predictions. This is the reason why we ask whether complex machine learning models are even necessary to use. We envision that practical resource management systems need to first identify the extent to which simple solutions

can be effective, and resort to using machine learning to the extent that enables its practical use.

CCS CONCEPTS

- **Computer systems organization** → **Cloud computing**;
- **Computing methodologies** → *Machine learning*.

KEYWORDS

Cloud Computing, Resource Usage, Forecasting, Prediction, Persistent Forecast, Data Persistence, Machine Learning, Long Short Term Memory

ACM Reference Format:

Georgia, Christofidi, Konstantinos, Papaioannou, and Thaleia Dimitra, Doudali. 2023. Is Machine Learning Necessary for Cloud Resource Usage Forecasting?. In *ACM Symposium on Cloud Computing (SoCC '23)*, October 30–November 1, 2023, Santa Cruz, CA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3620678.3624790>

1 INTRODUCTION

Cloud computing environments and data centers often exhibit low resource efficiency, due to the users requesting more resources than using in practice and through suboptimal decisions of cluster management systems [6, 14, 20, 35, 39]. Their efficiency can be improved by leveraging robust forecasts of metrics such as the resource consumption over time, the utilization pattern, the lifetime and the size of the virtual machines [14], for example in autoscaling resources [10, 26, 31, 36], setting overcommitment policies [7, 41, 43] and consolidating VMs [17, 22].

Production-level resource management systems use combinations of lightweight prediction methods, to deliver a practical deployment. These methods range from simple formulas that combine historical information [5, 14, 15, 25, 29] or a combination of them [7]. In addition, a few production systems explore the lightweight use of machine learning models, using an ensemble of such models alongside simple sliding window algorithms [36] or for a small percentage of servers with complex patterns [34]. At the same time, we see a rise in research artifacts that explore the use of machine

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SoCC '23, October 30–November 1, 2023, Santa Cruz, CA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0387-4/23/10...\$15.00
<https://doi.org/10.1145/3620678.3624790>

learning methods to predict resource utilization, using methods such as Convolutional Neural Networks (CNNs) [33], Gated Recurrent Units (GRUs) [40], Temporal Convolutional Networks (TCNs) [12] and Ensemble Learning [19].

In particular, Long Short Term Memory (LSTM) networks have shown great potential to reach higher accuracy than traditional methods in time series forecasting. For the problem of generic time series forecasting, new algorithms continuously arise, such as N-BEATS [32] and N-Hits [11]. Yet, in system-level resource management, we predominantly see solutions that use LSTMs. These have shown to deliver highly accurate forecasts when predicting cloud resource usage [10, 13, 18, 24, 37, 42, 52], memory access patterns [16, 21], the lifetime of memory objects [27], the system state under a certain execution environment [28] and the maximum load in inference serving systems [37].

Given the wide adoption of LSTMs in system-level resource management, we also use them to forecast resource usage over time for cloud-based workloads. We observe that LSTMs deliver high prediction accuracy, even when tested against workloads not seen during training for experimentation purposes. However, after visualizing the predicted time series compared to the original ones, we clearly observe that the predicted values resemble the original ones shifted by one time step in the future. We extensively look at related works and blog posts regarding the performance of LSTMs and confirm that their visualizations show similar shifts, which remain unnoticeable to most authors.

Thus, motivated by the observation that LSTMs predict a shifted version of the original data, we explore the effectiveness of a very simple prediction model that forecasts at time $t + 1$ the value at time t . Indeed, our **insight** is that such a simple model delivers similar accuracy levels as LSTMs. We attribute the success of this simple model to the fact that cloud resource usage time series data are highly correlated in time, meaning that the values between consecutive time steps tend to be close [2, 14, 39].

To validate this observation, we perform an extensive analysis using public data from Alibaba, Azure, Google and Bitbrains, which capture time series of various resource usage metrics, such as CPU, memory, network and disk traffic, as well as resource entities such as workloads, virtual and physical machines. We observe that consecutive values in such time series change by only 3.29%, on average. In such cases, the simple persistent forecast, mentioned above, can deliver sufficiently accurate forecasts. The acceptable levels of prediction accuracy and the need to use more complex or machine learning forecasting models will depend on how the quality of predictions translates to the end-to-end performance. For example, lower latency in autoscaling containers of microservices [26] or higher resource savings when setting overcommitment policies [7].

The **goal** of this paper is to debunk the necessity of complex, black-box machine learning models and draw attention to the potentially high effectiveness of using simple yet insightful solutions. Our analysis makes a strong case about using machine learning to the extent that is necessary, with a focus on drawing insights from the data itself. We showcase how a simple prediction mechanism, that leverages the unique characteristics of time series data, can deliver robust forecasts for the average case. Of course, there will always be cases that require a more sophisticated approach, such as machine learning-based decisions, and we need to develop mechanisms that identify such cases. We **envision** that machine learning should not replace, rather complement existing simple and practical solutions with the necessary amount of intelligence [16, 34].

The rest of the document is organized as follows. Section 2 contains the analysis that shows how LSTM-based predictions compare against simpler approaches and reveals the primary insight of the paper. Section 3 validates our insight across datasets and resource usage metrics. It summarizes important lessons learned from the analysis. Finally, the paper concludes by highlighting certain related works, in Section 4.

2 MOTIVATION

In this section, we evaluate the predictive capabilities of LSTMs, a popular type of machine learning model used for forecasting time series data. In addition, we explore their performance against different workloads, which reveals interesting insights.

2.1 Motivational Experiments

Experimental Setup. We use the 2019 Google Traces [44, 47], whose details are summarized in Section 3. The dataset consists of jobs that contain many tasks running under a set of resources.

For the deployment of the Long Short Term Memory (LSTM) neural networks, we train a different model with the same hyperparameters at the granularity of a job. All models have one layer of 50 interconnected LSTM units with a softmax activation function, followed by a fully connected layer. During training, we use the Adam optimizer, the Mean Absolute Error (MAE) as a loss function and set the history window to 30 time steps.

The combination of the above hyperparameter values is the one that delivered the lowest training error after fine-tuning. In summary, we experimented with [10, 50, 100, 500] LSTM units per layer, [1, 2, 4] stacked layers, [1, 5, 10, 30, 50] length of the history window, [relu, softmax, tanh and sigmoid] activation functions, [SGD, RMSprop, ADAM, Adadelta, Adagrad, Nadam] optimizers and the [Mean Squared

Error (MSE), Mean Absolute Error (MAE), Mean Squared Logarithmic error, Huber loss] for the loss function.

The training is set to stop at one thousand (1000) epochs, with the patience set at 20 epochs. The training dataset for each model corresponds to the time series of CPU usage for one randomly selected task of a job. The time series values are normalized to the $[0,1]$ interval using a linear transform, that performs min-max normalization before being introduced to the model. After inference, the predicted values are denormalized using the inverse transform.

Experiments. We conduct three sets of experiments, that define different test datasets to use during inference over the trained models, where there is one model per job. In more detail:

- **Experiment A** defines as test dataset the data used during training, to evaluate the achieved prediction accuracy per model.
- **Experiment B** uses as test data the time series of 10 randomly selected tasks of the same job, excluding the training data, running inference with the model corresponding to the same job.
- **Experiment C** uses the trained model of the job with identifier 113, to run inference against test data of 10 tasks belonging to the other jobs, namely 374, 399, 917 (x-axis). Experiment A captures the common practice in training machine learning models. Experiments B and C explore, for experimentation purposes, the accuracy of the models when predicting unseen data, that correspond to other tasks of the same job and completely different jobs, respectively. These two experiments are conducted to capture and showcase the main insight of this paper and highlight the misleading effectiveness of the LSTMs, to be discussed in Section 2.2. This is why the scale of the LSTM-based experiments is much smaller than the ones in Section 3, where we extensively evaluate the effectiveness of our proposed approach.

ML-based Forecasting. Figure 1 shows the mean absolute error (MAE $\times 100\%$) of the predicted vs. ground truth values (y-axis), when using LSTM models to forecast the resource usage across time of the corresponding jobs (x-axis). The non-ML prediction and corresponding results will be explained in Section 2.2. Next, we summarize our observations per experiment.

First, in experiment A we observe very low prediction error across all models, that is 2.8% on average, when we test on data already seen during training. This validates the high predictive capabilities of LSTM models over cloud workloads [8, 9, 13, 48].

Second, the experiment B captures the average MAE across different tasks of the same job and its standard deviation across tasks (shown as whiskers). Similar to experiment A the average error is very low, at 2.9%. This shows that the LSTM

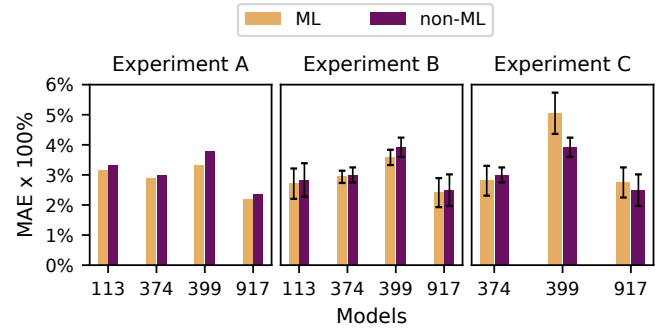


Figure 1: Resource usage prediction error when using ML models (LSTMs) vs. a non-ML prediction.

models can generalize across the tasks of a job, when using the data of only one task for training. This is not surprising because the tasks that belong to a single job tend to share similar patterns in resource usage [44].

Finally, experiment C shows the average MAE when using the trained model of job 113 to infer the tasks of the other jobs, with the standard deviation shown as whiskers. Surprisingly, the prediction error is still very low, on average 3.5%, and as high as 5%. The patterns across jobs can be very different [44], yet the LSTMs seem to deliver highly accurate forecasts even in this case. This behavior is very unexpected and makes us question the overall effectiveness of LSTMs in the context of forecasting cloud resource usage. Section 2.2 gives more insight behind the observed behavior of the LSTMs.

2.2 Insight on Misleading ML Effectiveness

To better understand the ML-based predictions, we visualize in Figure 2 part of the time series of CPU usage (y-axis) across time (x-axis) for job 113. Figure 2a compares the predicted time series to the ground truth. Each box depicts different parts of the time series, where the predicted one is derived following the three different experiments described above, using the trained model of job 113 for experiments A, B and job 917 for experiment C. The number on the upper right of each box shows the Pearson correlation coefficient value between the two depicted time series.

The insight. The Pearson correlation values are very close to the maximum value of 1, indicating that the predictions closely follow the ground truth. This also validates the low prediction errors depicted in Figure 1. However, a visual inspection reveals that the time series values predicted by the LSTM models follow a distinct shift on the time axis, towards the future, compared to the ground truth. In other words, the predicted values at any time step tend to follow very closely the value of the original time series in the *previous* time step. This is observed across all three experiments shown here and many others we have performed alongside.

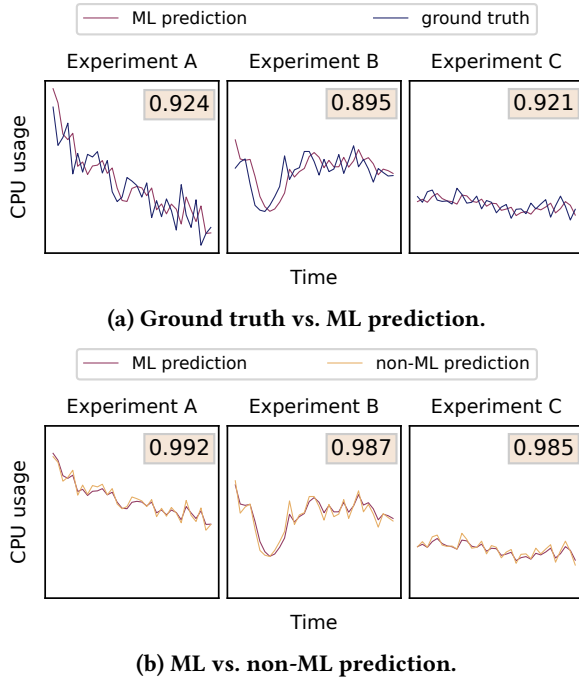


Figure 2: Visual example of the predicted time series.

To validate our observation, we revisit publications of related works, tutorials and blog posts and take a closer look at the illustrations used to show the effectiveness of the LSTMs. InfAdapter [37] uses LSTMs to forecast maximum load for improving inference serving systems. Figures 5 and 8 in that paper validate our insight, that the predicted values resemble the original values just shifted in time. Next, when LSTMs are used to predict time series of global active power consumption [4], we can observe in the last figure, a shift of the LSTM predictions to the future. Yet, the authors fail to notice that and conclude that "LSTMs are amazing". Similarly, we observe such shifted predictions when using LSTMs to forecast multidimensional time series of gas concentration values [49] and price movements in the financial domain [51].

Last but not least, we did find a blog post [2] that validates our observations. They attribute this behavior to the fact that time series data tend to be highly correlated in time, meaning that the value at time $t + 1$ is very likely to be close to the value at the previous time step t . In such cases, a very simple prediction model, namely the *persistent* forecast [34], can produce highly accurate predictions by assuming that $y(t + 1) = y(t)$, for each value y in the time series. We refer to this as the **non-ML prediction** in the analysis above. Figure 2b visually validates that the LSTM prediction indeed follows very closely the non-ML prediction. In other words, the LSTM predicts a value almost the same as the original value in the *previous* time step, as also confirmed by the Pearson correlation value being almost 1.

Revisiting Figure 1, we see that using the non-ML predictions (i.e., the persistent forecast) deliver almost the same MAE as the LSTM-based ones, with low standard deviation across tasks. This is a very powerful observation. It indicates that cloud resource usage values have gradual change over time, enabling a very simple solution to provide high accuracy levels, comparable to the ones possible with machine learning. This insight raises the following question: *is machine learning even necessary to deliver highly accurate cloud resource forecasts?*

Takeaways. The analysis above reveals that the high accuracy of LSTMs is enabled by predictions that resemble the original values shifted in time. We make a case how a very simple model, namely the persistent forecast, can deliver similar accuracy compared to more complex machine learning ones. Can such a simple forecasting model deliver accurate forecasts at scale, regardless of the resource usage metric, entity and cloud provider? The remainder of the paper explores the extent to which a simple non-ML cloud resource forecasting model can be effective.

3 ANALYSIS

In this Section, we explore in depth whether cloud resource usage time series are highly correlated in time, thus exhibit high data persistence over time. In practice, this means that values in consecutive time steps have a very small difference. In such cases, machine learning-based forecasting may not be necessary and simple predictive models can deliver the desired levels of accuracy [34].

3.1 Experimental Methodology

The code for all experiments in this paper is open-sourced.¹

Datasets. We use the publicly available Alibaba Cluster Traces (2018) [1, 20], the Azure VM Traces [3, 14], the Google Cluster Workload Traces (2019) [44, 47] and the Bitbrains Trace [39] datasets. Table 1 summarizes the characteristics of the different datasets. These include the resource type and entity, the number of available traces we consider and their length, and the time granularity at which measurements are collected.

The Alibaba dataset captures the CPU, memory, network traffic and disk I/O usage at the granularity of a physical machine. The sampled values are recorded approximately every one minute. We use part of the trace that captures the resource usage of 1000 machines during an 8-day period [1].

Furthermore, we use the 2019 version of the Azure VM Traces dataset, which measures the minimum, the maximum and the average CPU utilization per virtual machine, on a series of 5 minute intervals over a three-month period

¹<https://github.com/muse-research-lab/cloud-forecast-data-persistence>

Table 1: Cloud resource usage data used in experiments.

Dataset	Resource Type	Abbreviation	Resource Entity	Count	Avg Length	Frequency
Alibaba	CPU, Memory %	cpu, mem	Physical Machine	1000	6811	~1min
	In,Out network traffic %	net-in, net-out				
	Disk I/O %	disk-io				
Azure	Min, Max, Avg CPU %	{min,max,avg}-cpu	Virtual Machine	1195	7678	5 min
Google	Avg, Max CPU %	{avg,max}-cpu	Job Task	1635	4667	5 min
	Avg, Max memory %	{avg,max}-mem				
Bitbrains	CPU MHZ, %	cpu-raw, cpu mem-raw disk-{rd,wr} net-{recv,xmit}	Virtual Machine	2741	8272	5 min
	Memory KB					
	Disk reads, writes KB/sec					
	Network received, transmitted KB/sec					

[3]. The entire dataset consists of tens of millions of virtual machines from thousands of users [14], from which we use 1195 time series of CPU usage.

The Google dataset contains traces of workloads that consist of jobs. These jobs contain many tasks that run under a set of resources [44, 45, 47]. The trace records the average and maximum usage of CPU and memory reported every 5 minutes for one month. Due to the massive size of the dataset, we downsize to 1635 time series. The entire dataset contains traces of 8 different Borg cells, with a median cell size of 10 thousand machines.

Last, we use the Bitbrains dataset extracting 2741 time series, from more than five million CPU hours spanned across 1750 virtual machines over four months, that are available in the entire dataset [39]. More precisely, we use the average CPU usage over the sampling interval of 5 minutes, measured in MHZ and percentage, the actual memory usage reported in KB, the disk and the network I/O throughput, both in terms of KB/s.

Data Persistence Metric. To capture whether data values persist over time, we define a metric to associate with a time series, which we refer to as Average Relative Delta (ARD).

$$ARD = \frac{\sum_{t=1}^N |y_t - y_{t-1}|}{N} \times 100\%$$

To calculate the ARD, we first normalize all the values of a time series to the $[0,1]$ interval, using a min-max linear transform. This allows for a *relative* comparison across time series. Then, we smoothen the outliers, by adjusting 1% of the lowest and the highest values to the 1st and 99th %ile values accordingly. Next, we sum up the *absolute* deltas between two values y in consecutive time steps, that is $y_t - y_{t-1}$. Finally, we divide by the number of total time steps, to give an *average* number.

The purpose of this metric is to allow the comparison of different time series that represent different resource usage

metrics *across* datasets. The formula above resembles the Mean Absolute Error (MAE), which captures the prediction error in a forecasting model. In contrast, ARD aims to capture the degree of data persistence in a time series in an easy-to-understand and application-agnostic way. A similar approach for a persistent forecast was used in Seagull [34] to detect daily and weekly repeating patterns.

3.2 Experimental Results

Figure 3 shows the cumulative distribution of the average relative delta (ARD) metric per time series. The y-axis shows the probability of the ARD value being less than the corresponding value on the x-axis. The experiments are grouped by dataset and every line in the CDF plots corresponds to a different resource metric, as summarized in Table 1. We use red shades to illustrate the metrics related to CPU usage, green for the memory, blue for the network and black for the disk I/O. The star marks the average value of each distribution, and the dashed line marks the end of each line's tail. Next, we describe the most interesting observations per dataset.

Alibaba Dataset. For each resource metric in the Alibaba Dataset, the average ARD between the current and the previous time step is less than 7%, as we can see in the Figure 3a. We notice that the network traffic has almost zero ARD, with no tail in its distribution, because it has stable behavior over time [20]. The disk I/O traffic also shows small ARD, less than 4% for 95% of the samples. The memory's ARD is less than 5% for 99% of values, with no tail in the distribution, meaning no outliers, because the physical machines maintain high and stable memory utilization for days [20]. Although the ARD distribution of the CPU usage shows the largest ARD values, these are still relatively small, 6.97% on average, and less than 10% for 98% of all the time series. The physical machines use CPU with daily patterns [50] that range between 20% - 50% utilization on average across days [20].

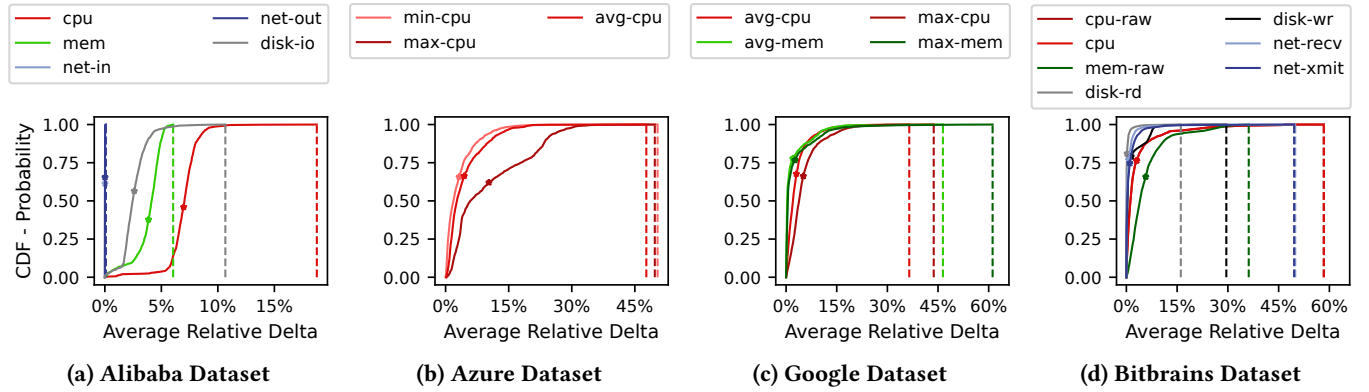


Figure 3: Cumulative distribution function of the average relative delta per timeseries across resources and datasets.

Azure Dataset. In this dataset, we are able to see any differences in the average relative delta of time series that capture the minimum, average and maximum CPU usage at the virtual machine-level. This distinction is important because there can be systems that do not predict average resource utilization, but rather the maximum one for overcommitment policies [7] and the minimum one for scheduling database backups [34]. Figure 3b shows that the max-cpu usage manifests larger ARD, 10.3% on average, compared to the min-cpu and avg-cpu that have an average ARD of only 3%. This is due to the fact that peak load tends to exhibit higher variance across time, compared to the average case [26, 50], and in general CPU usage is hard to predict with simple forecasting approaches [15].

Google Dataset. This dataset reports the average and maximum CPU and memory usage at the workload-level (jobs). Figure 3c shows that the CDF quickly increases, with the average ARD being less than 5.8% across all metrics. As also seen in the Azure dataset, the max-cpu and max-mem show slightly larger average ARD values compared to their average equivalents. Different from the Alibaba dataset, there is a smaller difference in the average ARD value of the CPU compared to the memory, where memory shows less pattern dynamicity than CPU usage [35]. Finally, we see that the distribution of the ARD values for both CPU and memory shows a big tail, where outliers can exhibit rather large ARD values. While we see similar tails in the CPU usage of the Azure dataset, this is not the case for Alibaba. This is due to the fact that the Google dataset captures resource usage at the job-level, incurring higher variability in the patterns across the time series [44].

Bitbrains Dataset. In Figure 3d, we observe that the distribution of the ARD of all the metrics has a rapid increase. The cpu-raw and the cpu lines are overlapping, both having a large tail, and an average ARD value of 3.05%. Here the memory usage shows larger average ARD, which can be

Table 2: Average relative and raw delta values for all the metrics in the Bitbrains Dataset.

Resource Type	Relative Delta	Raw Delta
CPU usage MHZ	3.05%	83.64 MHZ
Memory Usage KB	5.73%	129.63 MB
Disk reads KB/sec	0.21%	57.60 KB/sec
Disk writes KB/sec	1.41%	36.78 KB/sec
Network in KB/sec	0.66%	29.76 KB/sec
Network out KB/sec	1.03%	26.62 KB/sec

attributed to the fact that memory is measured in KBs, not percentage, resulting in larger differences between consecutive values. The ARD values of network and disk traffic show similar distribution to the Alibaba dataset, but with a significantly larger tail, since they capture bursty patterns at the virtual machine-level [39].

To relate the aforementioned *relative* delta values to actual differences in *raw* numbers, we summarize the average values for the resource usage metrics of the Bitbrains dataset in Table 2. We observe that the low values in the relative delta indeed correspond to small raw numbers, thus resulting in minimal e.g. disk and network traffic deltas. Even in the case of memory that shows the largest average delta of 5.73%, this corresponds to only few hundreds of MBs of memory used.

Sensitivity on the length of the time window. One could argue that cloud resource usage time series show very high correlation of values across time, because the values are recorded in very short windows of time, thus the small change in consecutive values is to be expected. As summarized in Table 1 the resource usage is reported every 1-5 minutes across datasets. What happens when we operate on a larger time window? Figure 4 shows the average ARD value across all time series of the corresponding resource metric (x-axis) for the Google dataset. Each bar corresponds to a different length of the time window, in which case we calculate the delta between the current value of the time

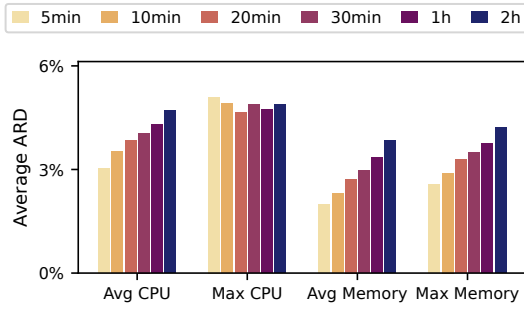


Figure 4: Average ARD when using different time windows for the delta calculation. Google dataset.

series and the one reported at the start of the corresponding time window.

We observe a very small difference in the average ARD values, that is around 2% between windows of 5 minutes to 2 hours. Different from the rest, the maximum CPU time series seems to be even less sensitive to the length of the time window, which can be explained by the fact that maximum measurements have higher variance in their ARD values, as also seen in Figures 3b, 3c. In general, we see that the cloud resource usage time series values show high data persistence even across large windows of time. This can be attributed to the fact that the patterns they capture are often periodic and follow diurnal cycles [44].

Time series of resource utilization exhibit high seasonality [15] and periodic patterns that can be hourly, daily, weekly [20, 39, 50]. According to Seagull [34] using the resource usage of the previous equivalent weekday or just the previous day as a prediction is more accurate than using the average value over that time window for weekly and daily patterns. Thus, using data shifts as predictions can be tuned to the time window that best captures the underlying patterns.

3.3 Discussion

Lessons learned. The above extensive analysis across various datasets, resource types and entities, shows that cloud resource usage time series data exhibit small delta between values in consecutive time steps, thus high data persistence over time. We conclude that:

- The level at which resource utilization is measured, plays an important role in the data persistence of the time series. Resource usage at the workload-level is more dynamic [44] than the virtual or physical machine-level, where levels of utilization are much more stable, periodic and predictable [15, 20, 34, 39, 50].
- The type of resource also contributes to different levels of data persistence. For example, average CPU usage usually exhibits periodic and daily patterns at the virtual [14, 39] and physical machine-level [20], whereas memory levels

are more stable over time [14, 20, 39], and disk and network traffic is bursty [39].

- Maximum measurements of resource usage show larger fluctuations in consecutive values, compared to the average and minimum measurements [26, 50].

Generalization to other datasets. The analysis in this paper spans across widely used datasets from various cloud providers, resource types and levels of use. Our proposed ARD metric can be applied at any dataset with information across time to capture data persistence, whose degree will indicate the potential effectiveness of a simple forecasting model. Since persistence is a property unique to time series data, it needs to be examined on a per-dataset basis.

Our extensive experimental evaluation enriches existing observations [34] which show the high effectiveness of simple persistent forecasts for cloud resource usage data that exhibit stable and periodic patterns across time. Thus, we believe that this can apply to other datasets with similar properties. For example, ML workloads exhibit periodic patterns in the number of task submissions and the amount of requested resources across time [46]. In addition, microservices typically demonstrate daily and weekly repeating patterns of how many Calls Per Minute (CPM) they receive due to regular user activity [26] and, similarly, serverless workloads in their function invocations [38]. Finally, memory access patterns, as observed at the Operating System (OS) page-level, are known to have high data persistence [16, 23, 30], since consecutive memory accesses fall into the same OS-level page, allowing the mix of persistent and ML predictions [16].

Impact of resource usage prediction to application-level performance. Table 3 presents several use cases where high prediction accuracy of future resource usage improves application-level metrics. For example, Seagull [34], a database backup scheduling system, uses the persistent forecast to predict the time windows when a server (virtual machine) has low (minimum) resource usage. This allows the reduction in backup collisions with the customer peak activity and improves customer experience. In addition, overcommitment policies with highly accurate predictions of per-machine peak resource usage enable significantly higher resource savings [7]. Moreover, microservice autoscaling systems deliver higher resource savings and reduced end-to-end service latency, when accurately forecasting the number of requests (calls) per minute to the microservices [26]. Similarly, workload autoscaling systems also reduce the number of out-of-memory errors during task execution, when accurately predicting and setting the limit of resources given for the execution of a job task [36]. As observed each use case has distinct end-to-end performance targets, thus the quality of predictions needs to be evaluated against those in every case. In conclusion, robust and practical prediction models of

Table 3: Prediction impact on application-level performance metrics in real world use cases.

Use Case	Predicted Metric	Level	Performance Metrics
Database Backup Scheduling [34]	Min Resource Usage	Virtual Machine	↓ Backup Collisions
Overcommitment Policies [7]	Peak Resource Usage	Physical Machine	↑ Resource Savings
Microservice Autoscaling [26]	Requests per Minute	Microservice	↑ Resource Savings ↓ End-to-End Latency
Workload Autoscaling [36]	Resource Limit	Job Task	↑ Resource Savings ↓ OOM Errors

resource usage related metrics unlock new opportunities for improved user experience, higher application performance and increased savings in resource utilization.

When to use machine learning for forecasting cloud resource usage? The necessity and effectiveness of simple solutions vs. complex black-box machine learning models, will depend on the exact use case, the degree of data persistence, the accuracy requirements and how much the predictions directly influence performance and decision-making. Given the insights of this paper, we give the following guidelines:

- Revisit systems evaluated over the datasets included in our analysis. Will a persistent forecast provide sufficiently accurate predictions that improve end-to-end performance, while enabling a more practical solution?
- For other datasets, study extensively the degree of data persistence using our proposed methodology and explore the potential effectiveness of a persistent forecast.
- To decide when to use machine learning, identify the *data subset* for which a simple forecast does not deliver the desired end-to-end performance.
- To decide which machine learning method to use for the identified data subset, the choices are endless! We suggest the use of a method that allows its practical integration to the system with low overheads for training and inference. We advocate for the insightful and judicious use of machine learning in resource management systems, that leverage simple and lightweight mechanisms to the extent possible.

4 RELATED WORK

This Section summarizes the prediction models used for forecasting resource usage in production-level systems. We highlight any insight behind the choice of using of machine learning.

Facebook’s Prophet [15] predicts various time series data based on an additive model with seasonality. Prophet delivers accurate forecasts for time series with strong seasonality, but fails to do so for metrics with more residual errors, like CPU usage. The authors suggest the use of machine learning in such cases, yet do not make any concrete proposals.

Google’s Autopilot [36] proposes a recommender system for setting resource limits used in vertical and horizontal

autoscaling. Autopilot uses a universal sliding window algorithm alongside an ensemble of lightweight machine learning models that minimize a custom cost function per job. This approach is inspired by machine learning, while delivering a practical solution at scale.

Microsoft’s Seagull [34] is a system that automatically schedules database backups, when users have low activity, by predicting customer load using a mix of simple and ML-based prediction models. For long-lived servers with stable load, daily and weekly patterns they use the corresponding previous load for prediction, and use ML only for 4.2% of the servers with more complex patterns.

Similar to Seagull [34], we envision system-level resource management solutions that make insightful and minimal use of ML, to reach the necessary accuracy levels in the corresponding use case. Different from Seagull, our work is motivated by our insight on LSTM predictions. In addition, our analysis spans time series data across cloud providers, resource entities and types, making a case that simple predictive models can work across various use cases, not just scheduling database backups.

5 SUMMARY

This paper reveals an interesting insight, that shows how Long Short Term Memory (LSTM) networks used for forecasting cloud resource usage over time, seemingly just predict an almost shifted version of the original data. We take this insight and showcase how a very simple prediction model can deliver highly accurate forecasts across types of resources and entities, using data from various cloud providers. The key to success is to leverage the high correlation of time series data across time. We make a case about how the use of machine learning in system-level resource management needs to be well-justified and to the extent that is necessary.

ACKNOWLEDGMENTS

We thank the reviewers and our shepherd Yiwen Zhu for their constructive feedback. This work is part of the grants FJC2021-047102-I, TED2021-132464B-I00, PID2022-142290OB-I00, funded by MCIN/AEI/10.13039/501100011033, the European Union «NextGenerationEU»/PRTR and the ESF+.

REFERENCES

- [1] 2018. Alibaba Cluster Traces 2018. Technical report at <https://github.com/alibaba/clusterdata/blob/master/cluster-trace-v2018>.
- [2] 2018. How (not) to use Machine Learning for time series forecasting: Avoiding the pitfalls. <https://towardsdatascience.com/how-not-to-use-machine-learning-for-time-series-forecasting-avoiding-the-pitfalls-19f9d7adf424>.
- [3] 2019. Azure VM Traces. Technical report at <https://github.com/Azure/AzurePublicDataset>.
- [4] 2019. Time Series Analysis, Visualization & Forecasting with LSTM. <https://towardsdatascience.com/time-series-analysis-visualization-forecasting-with-lstm-77a905180eba>.
- [5] George Amvrosiadis, Jun Woo Park, Gregory R. Ganger, Garth A. Gibson, Elisabeth Baseman, and Nathan DeBardeleben. 2018. On the diversity of cluster workloads and its impact on research results. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 533–546. <https://www.usenix.org/conference/atc18/presentation/amvrosiadis>
- [6] Luiz Andr Barroso, Jimmy Clidaras, and Urs Hlzl. 2013. *The Data-center as a Computer: An Introduction to the Design of Warehouse-Scale Machines* (2nd ed.). Morgan & Claypool Publishers.
- [7] Noman Bashir, Nan Deng, Krzysztof Rzadca, David Irwin, Sree Kodak, and Rohit Jnagal. 2021. Take It to the Limit: Peak Prediction-Driven Resource Overcommitment in Datacenters. In *Proceedings of the Sixteenth European Conference on Computer Systems* (Online Event, United Kingdom) (*EuroSys '21*). Association for Computing Machinery, New York, NY, USA, 556–573. <https://doi.org/10.1145/3447786.3456259>
- [8] Jing Bi, Shuang Li, Haitao Yuan, Ziyang Zhao, and Haoyue Liu. 2019. Deep neural networks for predicting task time series in cloud computing systems. In *2019 IEEE 16th International Conference on Networking, Sensing and Control (ICNSC)*. IEEE, 86–91.
- [9] Jing Bi, Shuang Li, Haitao Yuan, and MengChu Zhou. 2021. Integrated deep learning method for workload and resource prediction in cloud systems. *Neurocomputing* 424 (2021), 35–48.
- [10] David Buchaca, Josep LLuis Berral, Chen Wang, and Alaa Youssef. 2020. Proactive Container Auto-scaling for Cloud Native Machine Learning Services. In *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*. 475–479. <https://doi.org/10.1109/CLOUD49709.2020.00070>
- [11] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza, Max Mergenthaler-Canseco, and Artur Dubrawski. 2022. N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting. arXiv:2201.12886 [cs.LG]
- [12] Wenyan Chen, Chengzhi Lu, Kejiang Ye, Yang Wang, and Cheng-Zhong Xu. 2021. RPTCN: Resource Prediction for High-dynamic Workloads in Clouds based on Deep Learning. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. 59–69. <https://doi.org/10.1109/Cluster48925.2021.00038>
- [13] Georgia Christofidi, Konstantinos Papaioannou, and Thaleia Dimitra Doudali. 2023. Toward Pattern-Based Model Selection for Cloud Resource Forecasting. In *Proceedings of the 3rd Workshop on Machine Learning and Systems* (Rome, Italy) (*EuroMLSys '23*). Association for Computing Machinery, New York, NY, USA, 115–122. <https://doi.org/10.1145/3578356.3592588>
- [14] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. 2017. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles* (Shanghai, China) (*SOSP '17*). Association for Computing Machinery, New York, NY, USA, 153–167. <https://doi.org/10.1145/3132747.3132772>
- [15] Mustafa Daraghme, Anjali Agarwal, Ricardo Manzano, and Marzia Zaman. 2021. Time Series Forecasting using Facebook Prophet for Cloud Resource Management. In *2021 IEEE International Conference on Communications Workshops (ICC Workshops)*. 1–6. <https://doi.org/10.1109/ICCWorkshops50388.2021.9473607>
- [16] Thaleia Dimitra Doudali, Sergey Blagodurov, Abhinav Vishnu, Sudhanva Gurumurthi, and Ada Gavrilovska. 2019. Kleio: A Hybrid Memory Page Scheduler with Machine Intelligence. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing* (Phoenix, AZ, USA) (*HPDC '19*). Association for Computing Machinery, New York, NY, USA, 37–48. <https://doi.org/10.1145/3307681.3325398>
- [17] Fahimeh Farahnakian, Tapio Pahikkala, Pasi Liljeberg, Juha Plosila, Nguyen Trung Hieu, and Hannu Tenhunen. 2019. Energy-Aware VM Consolidation in Cloud Data Centers Using Utilization Prediction Model. *IEEE Transactions on Cloud Computing* 7, 2 (2019), 524–536. <https://doi.org/10.1109/TCC.2016.2617374>
- [18] Yu Gan, Yanqi Zhang, Kelvin Hu, Dailun Cheng, Yuan He, Meghna Pancholi, and Christina Delimitrou. 2019. Seer: Leveraging Big Data to Navigate the Complexity of Performance Debugging in Cloud Microservices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (*ASPLOS '19*). Association for Computing Machinery, New York, NY, USA, 19–33. <https://doi.org/10.1145/3297858.3304004>
- [19] Jashwant Raj Gunasekaran, Cyan Subhra Mishra, Prashanth Thinnakaran, Bikash Sharma, Mahmut Taylan Kandemir, and Chita R. Das. 2022. Cocktail: A Multidimensional Optimization for Model Serving in Cloud. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 1041–1057. <https://www.usenix.org/conference/nsdi22/presentation/gunasekaran>
- [20] Jing Guo, Zihao Chang, Sa Wang, Haiyang Ding, Yihui Feng, Liang Mao, and Yungang Bao. 2019. Who Limits the Resource Efficiency of My Datacenter: An Analysis of Alibaba Datacenter Traces. In *Proceedings of the International Symposium on Quality of Service* (Phoenix, Arizona) (*IWQoS '19*). Association for Computing Machinery, New York, NY, USA, Article 39, 10 pages. <https://doi.org/10.1145/3326285.3329074>
- [21] Milad Hashemi, Kevin Swersky, Jamie Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning Memory Access Patterns. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 1919–1928. <http://proceedings.mlr.press/v80/hashemi18a.html>
- [22] Nguyen Trung Hieu, Mario Di Francesco, and Antti Ylä-Jääski. 2020. Virtual Machine Consolidation with Multiple Usage Prediction for Energy-Efficient Cloud Data Centers. *IEEE Transactions on Services Computing* 13, 1 (2020), 186–199. <https://doi.org/10.1109/TSC.2017.2648791>
- [23] Sudarsun Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. HeteroOS: OS Design for Heterogeneous Memory Management in Datacenter. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto, ON, Canada) (*ISCA '17*). Association for Computing Machinery, New York, NY, USA, 521–534. <https://doi.org/10.1145/3079856.3080245>
- [24] Md. Ebtidaul Karim, Mirza Mohd Shahriar Maswood, Sunanda Das, and Abdullah G. Alharbi. 2021. BHyPreC: A Novel Bi-LSTM Based Hybrid Recurrent Neural Network Model to Predict the CPU Workload of Cloud Virtual Machine. *IEEE Access* 9 (2021), 131476–131495. <https://doi.org/10.1109/ACCESS.2021.3113714>
- [25] Chengzhi Lu, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Guodong Yang, and Chengzhong Xu. 2023. Understanding and Optimizing Workloads for Unified Resource Management in Large

- Cloud Platforms. In *Proceedings of the Eighteenth European Conference on Computer Systems* (Rome, Italy) (*EuroSys '23*). Association for Computing Machinery, New York, NY, USA, 416–432. <https://doi.org/10.1145/3552326.3587437>
- [26] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Guodong Yang, and Chengzhong Xu. 2022. The Power of Prediction: Microservice Auto Scaling via Workload Learning. In *Proceedings of the 13th Symposium on Cloud Computing* (San Francisco, California) (*SoCC '22*). Association for Computing Machinery, New York, NY, USA, 355–369. <https://doi.org/10.1145/3542929.3563477>
- [27] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. 2020. Learning-Based Memory Allocation for C++ Server Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (*ASPLOS '20*). Association for Computing Machinery, New York, NY, USA, 541–556. <https://doi.org/10.1145/3373376.3378525>
- [28] Dimosthenis Masouros, Christian Pinto, Michele Gazzetti, Sotirios Xydias, and Dimitrios Soudris. 2023. Adrias: Interference-Aware Memory Orchestration for Disaggregated Cloud Infrastructures. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 855–869. <https://doi.org/10.1109/HPCA56546.2023.10070939>
- [29] Yuan Mei, Luwei Cheng, Vanish Talwar, Michael Y. Levin, Gabriela Jacques-Silva, Nikhil Simha, Anirban Banerjee, Brian Smith, Tim Williamson, Serhat Yilmaz, Weitao Chen, and Guoqiang Jerry Chen. 2020. Turbine: Facebook’s Service Management Platform for Stream Processing. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1591–1602. <https://doi.org/10.1109/ICDE48307.2020.00141>
- [30] Mitesh R. Meswani, Sergey Blagodurov, David Roberts, John Slice, Mike Ignatowski, and Gabriel H. Loh. 2015. Heterogeneous memory architectures: A HW/SW approach for mixing die-stacked and off-package memories. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. 126–136. <https://doi.org/10.1109/HPCA.2015.7056027>
- [31] Hiep Nguyen, Zhiming Shen, Xiaohui Gu, Sethuraman Subbiah, and John Wilkes. 2013. AGILE: Elastic Distributed Resource Scaling for Infrastructure-as-a-Service. In *10th International Conference on Autonomic Computing (ICAC 13)*. USENIX Association, San Jose, CA, 69–82. <https://www.usenix.org/conference/icac13/technical-sessions/presentation/nguyen>
- [32] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. 2020. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. arXiv:1905.10437 [cs.LG]
- [33] Soukaina Ouham, Youssef Hadi, and Arif Ullah. 2021. An Efficient Forecasting Approach for Resource Utilization in Cloud Data Center Using CNN-LSTM Model. *Neural Comput. Appl.* 33, 16 (aug 2021), 10043–10055. <https://doi.org/10.1007/s00521-021-05770-9>
- [34] Olga Poppe, Tayo Amuneke, Dalitso Banda, Aritra De, Ari Green, Manon Knoertzer, Ehi Nosakhare, Karthik Rajendran, Deepak Shankargouda, Meina Wang, Alan Au, Carlo Curino, Qun Guo, Alekh Jindal, Ajay Kalhan, Morgan Oslake, Sonia Parchani, Vijay Ramani, Raj Sellappan, Saikat Sen, Sheetal Shrotri, Soundararajan Srinivasan, Ping Xia, Shize Xu, Alicia Yang, and Yiwen Zhu. 2020. Seagull: An Infrastructure for Load Prediction and Optimized Resource Allocation. *Proc. VLDB Endow.* 14, 2 (oct 2020), 154–162. <https://doi.org/10.14778/3425879.3425886>
- [35] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. 2012. Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing* (San Jose, California) (*SoCC '12*). Association for Computing Machinery, New York, NY, USA, Article 7, 13 pages. <https://doi.org/10.1145/2391229.2391236>
- [36] Krzysztof Rzdca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. 2020. Autopilot: Workload Autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems* (Heraklion, Greece) (*EuroSys '20*). Association for Computing Machinery, New York, NY, USA, Article 16, 16 pages. <https://doi.org/10.1145/3342195.3387524>
- [37] Mehran Salmani, Saeid Ghafouri, Alireza Sanaee, Kamran Razavi, Max Mühlhäuser, Joseph Doyle, Pooyan Jamshidi, and Mohsen Sharifi. 2023. Reconciling High Accuracy, Cost-Efficiency, and Low Latency of Inference Serving Systems. In *Proceedings of the 3rd Workshop on Machine Learning and Systems* (Rome, Italy) (*EuroMLSys '23*). Association for Computing Machinery, New York, NY, USA, 78–86. <https://doi.org/10.1145/3578356.3592578>
- [38] Mohammad Shahradd, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 205–218. <https://www.usenix.org/conference/atc20/presentation/shahradd>
- [39] Siqi Shen, Vincent Van Beek, and Alexandru Iosup. 2015. Statistical Characterization of Business-Critical Workloads Hosted in Cloud Datacenters. In *2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. 465–474. <https://doi.org/10.1109/CCGrid.2015.60>
- [40] Wenjuan Shu, Fanping Zeng, Zhen Ling, Junyi Liu, Tingting Lu, and Guozhu Chen. 2021. Resource Demand Prediction of Cloud Workloads Using an Attention-based GRU Model. In *2021 17th International Conference on Mobility, Sensing and Networking (MSN)*. 428–437. <https://doi.org/10.1109/MSN53354.2021.00071>
- [41] Xiaoyang Sun, Chunming Hu, Renyu Yang, Peter Garraghan, Tianyu Wo, Jie Xu, Jianyong Zhu, and Chao Li. 2018. ROSE: Cluster Resource Scheduling via Speculative Over-Subscription. In *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. 949–960. <https://doi.org/10.1109/ICDCS.2018.00096>
- [42] Xiaoyong Tang. 2019. Large-Scale Computing Systems Workload Prediction Using Parallel Improved LSTM Neural Network. *IEEE Access* 7 (2019), 40525–40533. <https://doi.org/10.1109/ACCESS.2019.2905634>
- [43] Huangshi Tian, Suyi Li, Ao Wang, Wei Wang, Tianlong Wu, and Hao-ran Yang. 2022. Owl: Performance-Aware Scheduling for Resource-Efficient Function-as-a-Service Cloud. In *Proceedings of the 13th Symposium on Cloud Computing* (San Francisco, California) (*SoCC '22*). Association for Computing Machinery, New York, NY, USA, 78–93. <https://doi.org/10.1145/3542929.3563470>
- [44] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhi-jing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. 2020. Borg: The next Generation. In *Proceedings of the Fifteenth European Conference on Computer Systems* (Heraklion, Greece) (*EuroSys '20*). Association for Computing Machinery, New York, NY, USA, Article 30, 14 pages. <https://doi.org/10.1145/3342195.3387517>
- [45] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-Scale Cluster Management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) (*EuroSys '15*). Association for Computing Machinery, New York, NY, USA, Article 18, 17 pages. <https://doi.org/10.1145/2741948.2741964>
- [46] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *19th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} '22)*.

- [47] J. Wilkes. 2020. Google Cluster Usage Traces v3. Technical report at <https://github.com/google/cluster-data>. Google.
- [48] Mahendra Pratap Yadav, Nisha Pal, and Dharmendar Kumar Yadav. 2021. Workload prediction over cloud server using time series data. In *2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, 267–272.
- [49] Tianjun Zhang, Shuang Song, Shugang Li, Li Ma, Shaobo Pan, and Liyun Han. 2019. Research on Gas Concentration Prediction Models Based on LSTM Multidimensional Time Series. *Energies* 12, 1 (2019). <https://doi.org/10.3390/en12010161>
- [50] Yongkang Zhang, Yinghao Yu, Wei Wang, Qiukai Chen, Jie Wu, Zuowei Zhang, Jiang Zhong, Tianchen Ding, Qizhen Weng, Lingyun Yang, Cheng Wang, Jian He, Guodong Yang, and Liping Zhang. 2022. Workload Consolidation in Alibaba Clusters: The Good, the Bad, and the Ugly. In *Proceedings of the 13th Symposium on Cloud Computing (San Francisco, California) (SoCC '22)*. Association for Computing Machinery, New York, NY, USA, 210–225. <https://doi.org/10.1145/3542929.3563465>
- [51] Svetoslav Zhelev and Dimitar R. Avresky. 2019. Using LSTM Neural Network for Time Series Predictions in Financial Markets. In *2019 IEEE 18th International Symposium on Network Computing and Applications (NCA)*. 1–5. <https://doi.org/10.1109/NCA.2019.8935009>
- [52] Yonghua Zhu, Weilin Zhang, Yihai Chen, and Honghao Gao. 2019. A novel approach to workload prediction using attention-based LSTM encoder-decoder network in cloud environment. *EURASIP Journal on Wireless Communications and Networking* 2019, 1 (2019), 274. <https://doi.org/10.1186/s13638-019-1605-z>